

Phase 2 — Technical Reference

phase2_optimiser.py — Bayesian Passive-Parameter Fitting Pipeline

Human Cortical Microcircuit Modelling Project

Overview

phase2_optimiser.py implements Bayesian optimisation of the three passive cable parameters (Cm, Rm, Ra) for multi-compartmental NEURON models of human cortical neurons, consuming the PassiveCell and OptimiserInputs objects produced by phase1_data_loader.py.

The module is designed to run on a standard laptop (sequential, development) or an HPC cluster (parallel across cells via multiprocessing.Pool). One process per (layer × cell type) group is the intended HPC deployment pattern.

Dependencies

Package	Purpose
scikit-optimize	gp_minimize, GP surrogate, use_named_args
numpy	Numerical operations
pandas	Results DataFrame
phase1_data_loader	SweepBundle, PassiveCell, OptimiserInputs, CellData
neuron (via Phase 1)	Compartmental simulation

Module-level constants

All defaults are defined at module level and can be overridden as keyword arguments on fit_one_cell and fit_cells.

```
python
```

```

DEFAULT_N_CALLS           = 150    # total gp_minimize evaluations per cell
DEFAULT_N_INITIAL         = 20     # random samples before GP guidance starts
DEFAULT_ACQ_FUNC           = "gp_hedge" # auto-selects EI/PI/LCB
DEFAULT_TRAIN_WINDOW_MS   = (1.0, 100.0) # Eyal 2016 fitting window
DEFAULT_VALID_WINDOW_MS_AFTER_ONSET = 50.0 # early Long Square window (ms)
DEFAULT_K_GOOD             = 3.0    # valid/train ratio threshold for "good"
DEFAULT_K_FAIL             = 10.0   # valid/train ratio threshold for "failed"
DEFAULT_TRAIN_RMSD_FAIL_MV = 2.0    # absolute train RMSD ceiling (mV)
DEFAULT_VALID_RMSD_GOOD_MV = 0.2    # absolute escape hatch (mV)
DEFAULT_UNCERTAINTY_DELTA_MV = 0.1  # GP-posterior plausibility band (mV)
DEFAULT_UNCERTAINTY_N_SAMPLES = 5000 # samples for uncertainty extraction

```

Log-transform of the parameter space

The optimiser works entirely in log-space. The convention throughout the codebase is:

```

q = log(p)    # variable the optimiser sees (natural logarithm)
p = exp(q)    # physical parameter passed to NEURON

```

The search space dimensions (defined in Phase 1's

`PassiveSearchSpace.as_skopt_dimensions()`) use the log-transformed bounds with a uniform prior in q :

Parameter	Physical bounds	$q = \log(p)$ bounds	Reason
Cm ($\mu\text{F}/\text{cm}^2$)	[0.3, 3.0]	[-1.204, 1.099]	Uniform prior appropriate for scale parameters
Rm ($\Omega\cdot\text{cm}^2$)	[1 000, 100 000]	[6.908, 11.513]	Two-decade range; log-uniform $\equiv$ uniform in log-space
Ra ($\Omega\cdot\text{cm}$)	[50, 1 000]	[3.912, 6.908]	One-decade range

Why log-transform? Four reasons:

1. **Scale normalisation.** In physical units the three ranges span vastly different absolute scales; the GP kernel would effectively ignore Cm and Ra against the 99 000 $\Omega\cdot\text{cm}^2$ range of Rm. In log-space all three ranges become scalars of order 1–5.

2. **Smoother landscape.** Passive cable transients depend on multiplicative combinations of the parameters ($\tau_m = C_m \times R_m$, $R_{in} \propto R_m$). These products are additive in log-space, making the loss surface closer to the quadratic shape the GP prior assumes.
3. **Consistent non-informative prior.** Uniform in $\log(p) = \log\text{-uniform in } p = \text{correct non-informative prior for scale parameters}$. Previously R_m had an explicit `prior="log-uniform"` while C_m and R_a used uniform, an inconsistency the log-transform resolves.
4. **Guaranteed positivity.** `exp(q) > 0` for all finite q ; the optimiser can never propose a non-physical value.

Conversion responsibility. The loss function (`_build_loss_function`) receives log-space values from `gp_minimize` and applies `np.exp()` before calling `cell.set_passive()`. The best-fit `result.x` from `gp_minimize` also contains log-space values and must be exponentiated before being stored in `PassiveFitResult`.

Uncertainty interpretation. `cm_sigma`, `rm_sigma`, `ra_sigma` are standard deviations in log-space. The approximate 68% physical confidence interval is:

```
p_lower = p_opt * exp(-sigma)
p_upper = p_opt * exp(+sigma)
```

A sigma of 0.3 corresponds to a multiplicative factor of $\exp(0.3) \approx 1.35$ in physical space.

Data model — `PassiveFitResult`

```
python
```

```

@dataclass
class PassiveFitResult:
    # Provenance
    specimen_id: int
    layer: str
    dendrite_type: str
    F: float          # spine-area correction factor applied
    fit_target: str    # "dep" | "hyp" | "both"

    # Fitted parameters
    cm_uF_per_cm2: float
    rm_0hm_cm2: float
    ra_0hm_cm: float

    # GP-posterior parameter uncertainties (one-sigma, surrogate-based)
    cm_sigma: float
    rm_sigma: float
    ra_sigma: float

    # Quality metrics
    train_rmsd_mV: float          # best-fit RMSD on training bundles
    valid_rmsd_mV: float          # equal-weight mean RMSD over all validation bundles
    valid_to_train_ratio: float   # valid_rmsd / train_rmsd

    # Allen reference scalars (cross-check only)
    rin_M0hm_allen: float
    tau_ms_allen: float
    v_rest_mV: float

    # Classification
    validation_status: str        # "good" | "to_refine" | "failed"
    n_calls: int
    n_initial: int
    wall_time_s: float
    error_message: str            # non-empty only on hard failure

    # Noise on averaged training traces (for Phase 3 / CI computation)
    noise_sigma_mV: float         # mean  $\sigma$  of pre-pulse baseline across training bundles
    noise_rho_lag1: float         # mean lag-1 autocorrelation
    noise_rho_lag1_high: bool     # True if  $\rho > 0.5$  (flag for  $n_{\text{eff}}$  correction)

    # Per-bundle validation breakdown (not written to parquet DataFrame)
    valid_rmsd_per_bundle: List[Tuple[str, float]] # [(label, rmsd_mV), ...]

```

```
n_validation_bundles: int

# Convergence trace (not written to parquet DataFrame)
loss_history: List[float]      # running minimum of loss vs. iteration
```

Parquet serialisation note. `loss_history` and `valid_rmsd_per_bundle` are list-valued fields that Arrow cannot serialise. `fit_cells()` drops both before building the DataFrame; they remain accessible on the in-memory `PassiveFitResult` objects in `results_list`.

Low-level helpers

`_crop_window(t, v, t_start_s, t_end_s) -> np.ndarray`

Returns the subset of `v` where `t` falls in `[t_start_s, t_end_s]`. No interpolation.

`_interp_to_grid(t_src, v_src, t_dst) -> np.ndarray`

Linear interpolation of `v_src` (defined on `t_src`) onto `t_dst`. Used before computing point-wise RMSD when NEURON's variable timestep produces a different time grid than the experimental trace.

`_baseline_subtracted_rmsd(t_exp_s, v_exp_mV, t_sim_s, v_sim_mV, pre_window_s, rmsd_window_s) -> float`

The fundamental quality metric.

1. Compute the mean of each trace over `pre_window_s` and subtract it (removes any Vrest offset between simulation and experiment).
2. Restrict both traces to `rmsd_window_s` on the experimental time grid.
3. Interpolate the simulated trace onto the experimental grid.
4. Return `sqrt(mean((v_exp - v_sim)^2))` in mV.

Returns `inf` if `rmsd_window_s` contains fewer than 2 experimental samples.

Protocol detection and labelling

`_is_brief_pulse(bundle) -> bool`

Returns `True` if `bundle.stim_duration_s < 0.01` s (i.e. Square Subthreshold, nominal 0.5 ms).
Returns `False` for Long Square (nominal 1 s). The 10 ms boundary is a safe gap — no Allen

protocol has a duration between 0.5 ms and 1 s.

This discriminator drives all per-bundle simulation and RMSD window dispatching.

`_bundle_label(bundle) -> str`

Returns a short diagnostic label: `"SS hyp (-200 pA)"` for a hyperpolarising Square Subthreshold bundle, `"LS (-90 pA)"` for a Long Square bundle.

Stimulus simulators

Both simulators return `(t_sim_s, v_sim_mV)` on the NEURON simulation grid.

`_simulate_square_subthreshold(cell, bundle, v_rest_mV, pre_pad_ms=10, post_pad_ms=100, dt_ms=0.025)`

Runs the model with an IClamp at the soma matching the bundle's amplitude and duration. The simulation starts `pre_pad_ms` before the pulse onset so the model has a settled baseline. Time is shifted so `t_sim=0` aligns with the bundle's pulse onset convention.

`_simulate_long_square(cell, bundle, v_rest_mV, dt_ms=0.1)`

Runs the model for the full duration of the Long Square sweep, using the bundle's `stim_onset_s` and `stim_duration_s` metadata to set the IClamp timing. Uses `dt_ms=0.1` (coarser than SS) because the Long Square step is 1 s long and finer timesteps give no additional information for the passive response.

Important. Both simulators use a single IClamp at amplitude `bundle.amplitude_pA` — the baseline-corrected step amplitude. The bias current present in the raw Allen recording (which maintains V_{rest}) is not replicated; instead, the model's V_{rest} is set via `e_pas`. This is correct because NEURON's `e_pas` already establishes the resting potential without requiring a holding current.

Noise estimator

`_estimate_noise(bundle, rho_threshold=0.5) -> (sigma_mV, rho_lag1, is_correlated)`

Measures the residual noise on the averaged trace from its pre-stimulus baseline segment.

Pre-window selection:

- Square Subthreshold (brief pulse): `t < 0` (the 10 ms pre-pulse baseline).
- Long Square: `[0, stim_onset_s - 5 ms]`. The 5 ms guard prevents onset transient bleed.

sigma_mV: standard deviation (ddof=1) of the pre-window voltage samples. This is the noise floor of the averaged trace. For a trace averaged over N pulses, σ decreases as $1/\sqrt{N}$ relative to the single-pulse noise.

rho_lag1: lag-1 autocorrelation, computed as $\frac{E[s[t]*s[t-1]]}{E[s[t]^2]}$ on mean-centred samples. Allen's recordings are acquired at 50 kHz through a 10 kHz Bessel filter; consecutive samples are correlated at the original sampling rate. After averaging, this correlation is reduced but may not be zero.

is_correlated: flag set to `True` when $\text{rho_lag1} > \text{rho_threshold}$. When `True`, the effective sample size for chi-squared calculations is reduced by approximately $n_{\text{eff}} = n * (1 - \text{rho}) / (1 + \text{rho})$. Phase 3's profile-likelihood CI computation should apply this correction.

Returns `(nan, nan, False)` if the pre-window contains fewer than 10 samples.

Per-bundle validation

`_rmsd_for_validation_bundle(cell, bundle, v_rest_mV, valid_window_ms_after_onset)`
`-> float`

Dispatches simulation and RMSD window based on protocol type:

Protocol	Simulator	pre-window	RMSD window
Square Subthreshold	<code>_simulate_square_subthreshold</code>	<code>[-10 ms, 0]</code>	<code>[1 ms, 100 ms]</code>
Long Square	<code>_simulate_long_square</code>	<code>[0, stim_onset_s]</code>	<code>[0, stim_onset_s + 50 ms]</code>

The Long Square RMSD window is deliberately restricted to the early 50 ms after step onset to minimise I_h contamination. I_h kinetics in human cortical neurons have activation time constants of 50–200 ms; within the first 50 ms the response is predominantly passive.

Loss function builder

`_build_loss_function(cell, train_bundles, v_rest_mV, train_window_ms, pre_window_ms=(-10, 0))` `-> Callable`

Returns a closure `loss(cm_log, rm_log, ra_log) -> float` suitable for `gp_minimize`.

The closure receives **log-space** values from the optimiser. Its first action is to convert to physical units:

```
python

cm = np.exp(cm_log)
rm = np.exp(rm_log)
ra = np.exp(ra_log)
```

Then for each call:

1. `cell.set_passive(Cm=cm, Rm=rm, Ra=ra)` applies cable parameters with per-segment F-factor scaling.
2. `cell.set_e_pas(v_rest_mV)` sets the reversal potential.
3. For each training bundle: simulate SS pulse, compute `_baseline_subtracted_rmsd` over `train_window_ms`.
4. Return the mean RMSD over all training bundles.

Any NEURON exception returns `1e6` (large but finite) and issues a warning. This prevents a single problematic parameter combination from crashing the entire `gp_minimize` call.

GP-posterior uncertainty extraction

`_gp_parameter_uncertainty(optim_result, delta_mV, n_samples, seed) -> dict`

Extracts per-parameter uncertainty from the trained GP surrogate without additional NEURON calls.

Algorithm:

1. Draw `n_samples` (default 5,000) uniform samples from the search space using `space.rvs(random_state=int(seed))`.
2. Transform to the GP's internal space via `space.transform(samples)`.
3. Predict loss at all samples using `gp.predict(samples_trans)`.
4. Keep samples whose predicted loss $\leq$ `opt_loss + delta_mV` (default 0.1 mV).
5. Return the standard deviation along each parameter axis of the kept samples.

Compatibility note. `space.rvs` requires `random_state` to be a plain `int`, `None`, or the legacy `numpy.random.RandomState`. Passing a `numpy.random.Generator` (the modern API) raises `ValueError` inside `sklearn.utils.check_random_state`. Phase 2 always passes `int(seed)`.

Failure handling. The function never raises. If `optim_result.models` is absent, if `space.rvs` fails, or if fewer than 5 plausible samples are found (surrogate too peaked), it returns `{"cm_sigma": nan, "rm_sigma": nan, "ra_sigma": nan}` or `{"cm_sigma": 0.0, ...}` respectively with a warning.

Validation status classifier

`_classify_fit(train_rmsd, valid_rmsd, k_good, k_fail, train_rmsd_fail_mV, valid_rmsd_good_mV) -> str`

Logic applied in strict priority order:

```
1. if not finite(train_rmsd) or train_rmsd > train_rmsd_fail_mV:
    return "failed"                                # training itself poor

2. if not finite(valid_rmsd):
    return "failed"                                # validation simulation crashed

3. if valid_rmsd <= valid_rmsd_good_mV:
    return "good"                                  # absolute escape hatch

4. ratio = valid_rmsd / max(train_rmsd, 1e-9)

5. if ratio <= k_good:    return "good"
   if ratio <= k_fail:    return "to_refine"
   else:                  return "failed"
```

Escape hatch rationale. When training RMSD is extremely small (e.g. 0.007 mV as observed for specimen 614635228), even a physically excellent validation RMSD of 0.083 mV produces a ratio of 11.9, exceeding the default `k_fail=10`. The absolute threshold `valid_rmsd_good_mV=0.2 mV` prevents this misclassification: a fit with sub-0.2 mV error on a several-millivolt deflection is biophysically trustworthy regardless of ratio.

Primary API

```
fit_one_cell(cell, cell_data, opt_in, *, F=1.9, n_calls=150, n_initial=20,  
acq_func="gp_hedge", valid_window_ms_after_onset=50, k_good=3.0, k_fail=10.0,  
train_rmsd_fail_mV=2.0, valid_rmsd_good_mV=0.2, uncertainty_delta_mV=0.1,  
uncertainty_n_samples=5000, seed=0, verbose=True) -> PassiveFitResult
```

Fits one cell end-to-end.

Execution sequence:

1. Defensive check: abort with status="failed" if train_bundles is empty.
2. Infer fit_target from polarities of train_bundles.
3. Build loss function (closure over cell + training bundles).
4. Decorate loss with @use_named_args to match gp_minimize's calling convention.
5. Call gp_minimize(n_calls=n_calls, n_initial_points=n_initial, acq_func=acq_func, random_state=seed).
6. Extract best-fit (cm_opt, rm_opt, ra_opt) from result.x.
7. Estimate noise: call _estimate_noise on each training bundle.
8. Validate: call _rmsd_for_validation_bundle for each validation bundle; compute equal-weight mean.
9. Classify: call _classify_fit.
10. Extract GP-posterior uncertainties: call _gp_parameter_uncertainty.
11. Assemble and return PassiveFitResult.

All failures at any step are caught. Outer failures (gp_minimize crash) return `status="failed"` with `error_message`. Inner failures (single validation bundle crash) are warned and that bundle is skipped; the remaining bundles still contribute to the mean.

```
fit_cells(cells_data, opt_inputs, *, cells=None, F=1.9, n_workers=1,  
return_results=True, **fit_kwargs) -> (DataFrame, List[PassiveFitResult])
```

Batch driver for an entire (layer × cell type) group.

Sequential mode (`n_workers=1`):

- Rebuilds NEURON model from SWC if `cells` list not provided.
- Calls `fit_one_cell` for each cell in order.
- Catches unexpected exceptions per-cell (returns `status="failed"` row) so the batch never aborts.

Parallel mode (`n_workers=N`):

- Uses `multiprocessing.Pool` with `spawn` context.
- Each worker calls `_fit_worker(cell_data, opt_in, F, kwargs)`, which imports Phase 1 locally and builds its own NEURON model. NEURON sections are process-global and cannot be pickled across processes; workers must each create their own.
- Results collected via `pool.imap_unordered`; ordering is not preserved.

Output DataFrame columns (27 total for `n_avg_groups=1`):

Column group	Fields
Provenance	<code>specimen_id</code> , <code>layer</code> , <code>dendrite_type</code> , <code>F</code> , <code>fit_target</code>
Fit	<code>cm_uF_per_cm2</code> , <code>rm_0hm_cm2</code> , <code>ra_0hm_cm</code>
Uncertainty	<code>cm_sigma</code> , <code>rm_sigma</code> , <code>ra_sigma</code>
Quality	<code>train_rmsd_mV</code> , <code>valid_rmsd_mV</code> , <code>valid_to_train_ratio</code>
Allen reference	<code>rin_M0hm_allen</code> , <code>tau_ms_allen</code> , <code>v_rest_mV</code>
Classification	<code>validation_status</code> , <code>n_calls</code> , <code>n_initial</code> , <code>wall_time_s</code> , <code>error_message</code>
Noise	<code>noise_sigma_mV</code> , <code>noise_rho_lag1</code> , <code>noise_rho_lag1_high</code>
Validation summary	<code>n_validation_bundles</code>

`loss_history` and `valid_rmsd_per_bundle` are dropped from the DataFrame but are preserved on the `PassiveFitResult` objects in the returned list.

```
aggregate_population(df, *, group_by=("layer", "dendrite_type"), accept_statuses=
("good",)) -> DataFrame
```

Computes per-group population statistics.

Groups the DataFrame by `group_by`, restricts to rows with `validation_status` in `accept_statuses`, and returns:

Output column	Meaning
<code>n</code>	number of accepted cells in the group
<code>cm_mean</code> , <code>cm_std</code>	Cm statistics ($\mu\text{F}/\text{cm}^2$)
<code>rm_mean</code> , <code>rm_std</code>	Rm statistics ($\Omega\cdot\text{cm}^2$)
<code>ra_mean</code> , <code>ra_std</code>	Ra statistics ($\Omega\cdot\text{cm}$)
<code>status_summary</code>	counts across ALL cells in the group (e.g. <code>"failed:1; good:5; to_refine:2"</code>)

`cm_std = 0` when only one cell is accepted (ddof=1 undefined for n=1).

`plot_fit_diagnostic(cell, cell_data, opt_in, result, figsize=(13, 9), show=True) -> (fig, axes)`

2x2 diagnostic figure. All panels apply the same best-fit parameters before drawing.

```
+-----+-----+
| Training                | SS held-out validation    |
| First train_bundle      | First _is_brief_pulse     |
| (fit polarity, ~±200 pA) | bundle in validation_bundles|
| Window: 1-100 ms (shaded) | (opposite polarity)      |
| RMSD = train_rmsd_mV    | RMSD from per_bundle dict |
+-----+-----+
| LS validation           | GP convergence            | | |
| First non-brief bundle  | Running min of func_vals  |
| in validation_bundles   | Log y-scale               |
| (smallest |amp| LS step) | Dotted line = n_initial   |
| Window: 0-(onset+50ms)  |                             |
| RMSD from per_bundle dict |                             |
+-----+-----+
```

Protocol dispatch inside the overlay helper: brief-pulse bundles call

`_simulate_square_subthreshold` and use `t < 0` as the pre-window; Long Square bundles call `_simulate_long_square` and use `t < stim_onset_s` as the pre-window. The baseline is

computed as the mean of the pre-window samples; if the pre-window is empty (which would cause a NaN model trace), it falls back to the first sample. This was the root cause of the original "Mean of empty slice" warning when the SS dep bundle was mistakenly treated as a Long Square.

Missing panel types (e.g. no SS held-out when `fit_target="both"`), or no Long Square bundles when `n_long_validation=0`) display a "no data for this panel" grey text rather than raising.

The figure supitle carries three lines: cell provenance, fitted parameters $\pm$ uncertainties, and the validation summary (mean RMSD, number of bundles, status).

Complete usage example

```
python
```

```

from phase1_data_loader import (
    list_human_cells_with_morphology,
    load_complete_cells,
    build_neuron_model,
    prepare_optimiser_inputs,
)
from phase2_optimiser import fit_cells, aggregate_population, plot_fit_diagnostic

# --- Phase 1 ---
candidates = list_human_cells_with_morphology(
    layer="6", dendrite_type="spiny")
cells_data = load_complete_cells(candidates)

# n_avg_groups=1: one bundle per polarity (high-SNR single average)
# n_avg_groups=K: K bundles per polarity for Phase 3 bootstrap uncertainty
opt_inputs = [
    prepare_optimiser_inputs(cd, fit_target="hyp", n_long_validation=None)
    for cd in cells_data
]

# --- Phase 2: sequential (development) ---
df, results = fit_cells(
    cells_data, opt_inputs,
    F=1.9,                # L6 spiny: Eyal default; update with H01 values
    n_workers=1,          # HPC: set to number of cores per node
    n_calls=150,
    n_initial=20,
)

# --- Inspect one cell ---
pc = build_neuron_model(cells_data[0].swc_path, F=1.9)
plot_fit_diagnostic(pc, cells_data[0], opt_inputs[0], results[0])
print(results[0].valid_rmsd_per_bundle)  # full per-bundle breakdown

# --- Population statistics ---
pop = aggregate_population(df, accept_statuses=("good",))
print(pop[["layer", "dendrite_type", "n", "cm_mean", "cm_std",
           "rm_mean", "rm_std", "ra_mean", "ra_std"]])

# --- Save (parquet-safe: list fields already dropped by fit_cells) ---
df.to_parquet("phase2_results_layer6_spiny.parquet")

```

HPC deployment pattern

One Slurm job per (layer $\times$ cell type $\times$ dendrite type) group:

bash

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=32
#SBATCH --time=04:00:00
#SBATCH --job-name=passive_fit_L6_spiny

python run_passive_fit.py \
    --layer 6 \
    --dendrite_type spiny \
    --n_workers 32 \
    --n_calls 150 \
    --output_dir /scratch/passive_fits/
```

python

```

# run_passive_fit.py
import argparse
from phase1_data_loader import *
from phase2_optimiser import *

parser = argparse.ArgumentParser()
parser.add_argument("--layer")
parser.add_argument("--dendrite_type")
parser.add_argument("--n_workers", type=int)
parser.add_argument("--n_calls", type=int, default=150)
parser.add_argument("--output_dir")
args = parser.parse_args()

candidates = list_human_cells_with_morphology(
    layer=args.layer, dendrite_type=args.dendrite_type)
cells_data = load_complete_cells(candidates)
opt_inputs = [prepare_optimiser_inputs(cd) for cd in cells_data]

df, _ = fit_cells(
    cells_data, opt_inputs,
    F=1.9,
    n_workers=args.n_workers,
    n_calls=args.n_calls,
)
out_path = f"{args.output_dir}/L{args.layer}_{args.dendrite_type}.parquet"
df.to_parquet(out_path)
print(f"Saved {len(df)} cells to {out_path}")

```

Known limitations and planned Phase 3 work

GP uncertainty is approximate. The surrogate-based sigma values are reliable only near the optimum where evaluations are dense. They should be treated as first-pass estimates. Phase 3 will implement profile-likelihood CIs using the GP to propose candidate boundary points and the real NEURON simulator to verify them, per the Bayesian Optimisation and Profile Likelihood notes.

Ih contamination in validation. The 50 ms early window reduces but does not eliminate Ih contamination in the Long Square validation. Cells with strong Ih (visible as a large voltage sag beginning within 50 ms of step onset) will show a systematic validation RMSD inflation that is a property of the active conductance, not of the passive fit. These cells are correctly labelled

"to_refine" and are candidates for Phase 3 refinement, where adding a phenomenological I_h to the model before refitting may be necessary.

Uniform F factor. The current implementation applies a single $F = 1.9$ to all dendrites beyond $60\text{ }\mu\text{m}$ from the soma, following Eyal et al. (2016). Layer-specific F values derived from the H01 connectome are a planned replacement for L4 and L6.

No bootstrap. The `n_avg_groups` infrastructure in Phase 1 supports bootstrap uncertainty by producing K independent average groups. Phase 2 uses all K training bundles simultaneously (mean RMSD loss) rather than fitting K times with held-out folds. Bootstrap refitting is deferred to Phase 3 as a secondary uncertainty cross-check.